ระบบหลังบ้าน SOML — ตัวนี้คืออะไร ใส่มาทำไม

เอกสารนี้อธิบายโค้ดใน `soml/backend/` ทีละโฟลเดอร์ ทีละไฟล์ ว่า **คืออะไร · ใส่มาทำไม · มาจากตรงไหนในบทที่ 3** เขียนให้อ่านคู่กับโค้ดใน VS Code — เปิดไฟล์ไหน ให้อ่านหัวข้อนั้น

ประโยคเดียวที่สรุปทั้งหลังบ้าน

> หลังบ้านคือ “ชั้นตรรกะทางธุรกิจ” ในตาราง 3.17 — ทุกกฎของร้านอยู่ที่นี่ หน้าจอเป็นแค่คนถาม ฐานข้อมูลเป็นแค่คนจำ

---

1. โครงทั้งหมด — อ่านตารางที่ 3.13 แล้วเห็นโฟลเดอร์

```
backend/
├── app.js                ประกาศเส้นทาง + สิทธิ์ ที่เดียว          ← ตาราง 3.18
├── server.js             เปิดพอร์ต ผูก WebSocket เริ่ม timer      ← หน่วยติดตั้งที่ 2
├── config/db.js          พูลการเชื่อมต่อ + ธุรกรรม               ← ตาราง 3.14 แถว 2
├── lib/promptpay.js      สร้าง QR ตาม EMVCo                          ← ตาราง 3.14 แถว 3
├── middleware/auth.js    ตรวจโทเคน + บทบาท คั่นหน้าทุกเส้นทาง     ← ตาราง 3.14 แถว 1
├── modules/              หนึ่งโฟลเดอร์ = หนึ่งโมดูลในตาราง 3.13
│   ├── auth              1  ส่วนควบคุมการเข้าสู่ระบบ
│   ├── product           2  ส่วนควบคุมข้อมูลสินค้า
│   ├── order             3  ส่วนควบคุมคำสั่งซื้อ
│   ├── queue             4  ส่วนจัดการคิวงาน
│   ├── dispatch          5  ส่วนควบคุมการจ่ายสินค้า
│   ├── stock-deduction   6  บริการตัดสต็อก
│   ├── dashboard         7  ส่วนรวบรวมข้อมูลแดชบอร์ด
│   ├── alert             8  ส่วนตรวจเงื่อนไขแจ้งเตือน
│   ├── audit-log         9  บริการบันทึกประวัติ
│   └── print-client     10  ส่วนเชื่อมต่อบริการพิมพ์
└── tests/                unit + integration                    ← ตาราง 3.22
```

ใส่มาทำไมถึงแบ่งแบบนี้ — ตาราง 3.13 ในเล่มมี 10 แถว โค้ดจึงมี 10 โฟลเดอร์ ชื่อเดียวกัน ลำดับเดียวกัน ถ้าอาจารย์ชี้แถวไหนในตาราง เปิดโฟลเดอร์นั้นได้ทันที ไม่ต้องค้น

กติกาที่ทุกไฟล์ทำตาม — ขึ้นต้นด้วยกล่องคอมเมนต์ 3 อย่าง: มาจากตารางไหน · รับผิดชอบเรื่องเดียวอะไร · ไม่ทำอะไร (หลัก Separation of Concerns ในบทที่ 2 หัวข้อ 2.1.1: หนึ่งโมดูลเปลี่ยนแปลงด้วยเหตุผลเดียว)

ประโยคที่ควรพูด — “ผมไม่ได้แบ่งโฟลเดอร์ตามชนิดไฟล์ (controllers/ services/ routes/) ครับ ผมแบ่งตามโมดูลในตาราง 3.13 เพราะเวลาอาจารย์ถามถึงโมดูลไหน ผมเปิดโฟลเดอร์เดียวแล้วเห็นทั้งหมด”

---

2. ส่วนสนับสนุน 3 ตัว — ไม่ใช่ตรรกะธุรกิจ แต่ทุกโมดูลพึ่ง

2.1 `config/db.js` — ที่เดียวที่รู้ว่าฐานข้อมูลอยู่ไหน

คืออะไร — ส่งออก 2 อย่าง: `pool` (การเชื่อมต่อร่วมกัน สูงสุด 10 ตาม NFR-06) และ `withTransaction(fn)`

ใส่มาทำไม — ตาราง 3.14 แถว 2 บอกว่าต้องมี "ส่วนตั้งค่าการเชื่อมต่อฐานข้อมูล จัดการพูล" ถ้าไม่มี แต่ละโมดูลจะเปิดการเชื่อมต่อเอง เปลี่ยนรหัสผ่านทีต้องแก้ 10 ที่

บรรทัดที่สำคัญที่สุด — `withTransaction` 6 บรรทัด: `beginTransaction → fn(conn) → commit` ถ้า fn โยน error → `rollback` แล้วโยนต่อ · `release()` อยู่ใน `finally` เสมอ

คำถามที่อาจารย์น่าจะถาม — "ทำไม release ต้องอยู่ใน finally" → ถ้าอยู่หลัง commit แล้ว fn โยน error การเชื่อมต่อจะไม่ถูกคืนพูล ครบ 10 ครั้งระบบค้างทั้งระบบ

ทดสอบพิสูจน์ (`tests/integration/db.test.js`) — โยน error กลางธุรกรรม → นับแถวก่อน-หลังเท่ากันเป๊ะ นี่คือ atomicity ในหัวข้อ 2.1.2 ที่จับต้องได้

2.2 `lib/promptpay.js` — แปลง (เบอร์ร้าน, ยอด) เป็นสตริง QR

คืออะไร — QR ไม่ใช่รูป มันคือสตริงตัวอักษรตามมาตรฐาน EMVCo ที่แอปธนาคารทุกแอปอ่านออก ไฟล์นี้สร้างสตริงนั้น หน้าจอเอาไปวาดเป็นภาพเอง

ใส่มาทำไม — ตาราง 3.14 แถว 3 และ FR-03 · ทำงานในกระบวนการเดียวกัน ไม่ต่อธนาคาร (ขอบเขต 1.3.6)

โครงของสตริง — TLV: `รหัสช่อง 2 หลัก + ความยาว 2 หลัก + ค่า` ต่อกัน เช่น `5303764` = ช่อง 53 ยาว 3 ค่า 764 (บาท) · ช่อง 63 ท้ายสุดคือ CRC-16 ของทุกอย่างก่อนหน้า ถ้าผิดแอปธนาคารสแกนไม่ขึ้น

คำถามที่อาจารย์น่าจะถาม — "รู้ได้ไงว่า CRC ถูก" → มีค่าอ้างอิงสากล `crc16("123456789") = 29B1` ทดสอบไว้ใน `promptpay.test.js` · "ทำไมช่อง 01 เป็น 12" → 12 = ใช้ครั้งเดียว เพราะเราผูกยอดไว้ใน QR ลูกค้าไม่ต้องพิมพ์ยอดเอง จึงไม่มีทางโอนผิดยอด

2.3 `middleware/auth.js` — ด่านตรวจหน้าทุกเส้นทาง

คืออะไร — 2 ฟังก์ชันคั่นหน้าเส้นทาง: `verifyToken` (ไม่มี/ปลอม/หมดอายุ → 401) และ `requireRole('manager', …)` (บทบาทไม่ตรง → 403)

ใส่มาทำไม — ตาราง 3.14 แถว 1 และย่อหน้าท้ายตารางบอกเหตุผลไว้เอง: "คั่นหน้าทุกเส้นทางในที่เดียว แทนที่จะเขียนตรวจซ้ำในแต่ละโมดูล … เมื่อเพิ่มเส้นทางใหม่ ผู้พัฒนาไม่สามารถลืมตรวจสิทธิ์ได้" (หลัก RBAC ใน 2.1.7)

401 กับ 403 ต่างกันอย่างไร — 401 = ไม่รู้ว่าคุณเป็นใคร · 403 = รู้ว่าเป็นใคร แต่บทบาทนี้ทำไม่ได้ · ทดสอบแยกกันทั้งสองกรณี

รายละเอียดที่ตั้งใจ — ไฟล์นี้โยน error ตอน require ถ้า `JWT_SECRET` ยังเป็น `change-me` → ระบบเปิดไม่ขึ้นเลย ดีกว่าเปิดขึ้นแล้วใครก็ปลอมโทเคนได้

---

3. `app.js` และ `server.js` — สองไฟล์ที่ประกอบทุกอย่างเข้าด้วยกัน

3.1 `app.js` — ตาราง 3.18 ในรูปโค้ด

คืออะไร — ไฟล์เดียวที่ประกาศว่า "เส้นทางไหน ใครเข้าได้ ไปเรียกฟังก์ชันอะไร" ทั้ง 17 เส้นทาง

```
app.post('/api/orders/:id/dispatch', verifyToken, requireRole('warehouse','manager'),
        dispatch.confirm);
//        เส้นทาง                    ใคร          บทบาทไหน                    ทำอะไร
```

ใส่มาทำไม — ถ้าอาจารย์ถาม "เส้นทางนี้ใครเข้าได้บ้าง" เปิดไฟล์เดียวตอบได้ครบ · โมดูลใน `modules/` **ไม่มีสิทธิ์ประกาศเส้นทางเอง** เป็นกฎที่ตั้งใจ

ประโยคที่ควรพูด — "app.js ของผมคือตาราง 3.18 บรรทัดต่อบรรทัดครับ เพิ่มแถวในตาราง = เพิ่มบรรทัดในไฟล์"

3.2 `server.js` — แค่เปิดพอร์ต

คืออะไร — 15 บรรทัด: สร้าง http server จาก app · ผูก WebSocket ของ queue · เริ่ม timer ของ alert · listen

ใส่มาทำไมต้องแยกจาก app.js — ชุดทดสอบใช้ `app` ผ่าน supertest โดยไม่ต้องเปิดพอร์ตจริง รันทดสอบ 93 ข้อได้ใน 3 วินาทีโดยไม่ชนกับระบบที่รันอยู่

---

4. โมดูลทั้ง 10 — ไล่ตามลำดับที่เรียกกัน

4.1 `modules/audit-log` (โมดูล 9) — ทำก่อนเพราะทุกคนเรียก

คืออะไร — `record(conn, event)` เขียนลง `audit_logs` และ `list` ให้ผู้จัดการค้น (UC-10)

ใส่มาทำไม — FR-09 ต้องบันทึกเหตุการณ์สำคัญทุกอัน เส้นประในรูปที่ 3.8 คือทุกโมดูลชี้มาที่นี่

การตัดสินใจสำคัญ — `record` **รับ** `conn` **จากผู้เรียก** ไม่เปิดเอง → ประวัติอยู่ในธุรกรรมเดียวกับงาน ถ้างาน rollback ประวัติหายตาม จึงไม่มีทางมี "ประวัติของสิ่งที่ไม่ได้เกิดจริง" (3.7.3 ขั้น 5)

รายชื่อเหตุการณ์ `ACTIONS` ประกาศไว้ที่เดียว 10 ชื่อ สะกดผิดถูกปฏิเสธทันที ไม่ให้ `LOGIN` กับ `LOGGED_IN` ปนกันในฐาน

4.2 `modules/auth` (โมดูล 1) — ออกโทเคน

คืออะไร — `POST /api/auth/login` เทียบ bcrypt → ออก JWT อายุ 8 ชั่วโมง (หนึ่งกะ) → บันทึก LOGIN · `GET /api/auth/me` คืนข้อมูลผู้ใช้จากโทเคน

ใส่มาทำไม — UC-01 · FR-01 · ฝั่ง “ออกโทเคน” อยู่ที่นี่ ฝั่ง “ตรวจโทเคน” อยู่ที่ middleware — คนละหน้าที่

การตัดสินใจสำคัญ — ชื่อผิด / รหัสผิด / บัญชีถูกปิด ตอบ**ข้อความเดียวกัน**หมด และเทียบ bcrypt แม้ไม่พบผู้ใช้ (กับ hash หลอก) → คนนอกใช้หน้าล็อกอินเดาไม่ได้ว่าชื่อไหนมีอยู่จริง ทั้งจากข้อความและจากเวลาตอบ

ทดสอบ — `auth.test.js` บล็อกสุดท้ายคือ TC-01 ตรง ๆ: ไม่แนบโทเคน / แนบโทเคนปลอม / แนบโทเคนพนักงานขายยิงเส้นทางผู้จัดการ → ถูกปฏิเสธทั้งสาม (AC-01)

4.3 `modules/product` (โมดูล 2) — สินค้าและการเปลี่ยนสต็อกที่ “คนทำ”

คืออะไร — ค้นหา (ทุกบทบาท) · เพิ่ม/แก้ (ผู้จัดการ) · **ปรับสต็อกด้วยมือพร้อมเหตุผล** (ผู้จัดการ)

ใส่มาทำไม — UC-02 · FR-02 · ตาราง 3.13 โมดูล 2 เขียนคำว่า “ปรับสต็อกด้วยมือพร้อมเหตุผล” ไว้ชัด

การตัดสินใจสำคัญ - `PUT` แก้ `stock_qty` ตรง ๆ **ไม่ได้** — ต้องผ่าน `PATCH /stock` เท่านั้น เพื่อให้ทุกการเปลี่ยนมี `reason` และ `balance_after` ใน `stock_movements` (3.7.2 ข้อ 2) ไม่งั้นไล่ย้อนไม่ได้ - `manual_adjust` บังคับมี `note` แต่ `receive` ไม่ต้อง — รับของเข้าเหตุผลชัดในตัว ส่วนปรับตามการนับต้องบอกว่าทำไมถึงไม่ตรง - ไม่มีลบ — ปิดขายด้วย `is_active` เพราะ `order_items` เก่ายังอ้างถึง - หลัง commit เรียก `alert.checkStock` — BR-05 “ตรวจเองหลังทุกครั้งที่สต็อกเปลี่ยน” การปรับด้วยมือก็คือสต็อกเปลี่ยน

4.4 `modules/print-client` (โมดูล 10) — เล็กที่สุด แต่มีสัญญาสำคัญ

คืออะไร — ฟังก์ชันเดียว `printReceipt(receipt)` ยิง POST ไปบริการพิมพ์ (หน่วยติดตั้งที่ 3)

ใส่มาทำไมต้องแยก — 3.4.1 บอกเอง: “เป็นส่วนเดียวที่ผูกกับอุปกรณ์ทางกายภาพ” · ไฟล์นี้ไม่รู้ว่าเครื่องพิมพ์ยี่ห้ออะไร รู้แค่ URL

สัญญา — ไม่โยน error เด็ดขาด คืน `{ok, error}` เสมอ ทั้งกรณีตอบ 500 / ไม่มีบริการ / ค้างเกินเวลา · เพราะ UC-03 ทางที่ล้มเหลว: “พิมพ์ไม่สำเร็จ คำสั่งซื้อยังต้องสำเร็จ” ถ้าโยน ผู้เรียกลืมดัก try/catch ครั้งเดียว การขายจะล้มเพราะกระดาษหมด

4.5 `modules/queue` (โมดูล 4) — คิวและช่องทางค้าง

คืออะไร — 2 ส่วนในไฟล์เดียว: (ก) `GET /api/queue` + `PATCH /status` (ข) `broadcast()` ผลักเหตุการณ์ผ่าน WebSocket + SSE

ใส่มาทำไม — UC-05, UC-06 · FR-04 · BR-03 “คลังต้องรู้ก่อนลูกค้าเดินไปถึง” → ต้องผลัก ไม่ให้หน้าจอถามเอง

ตาราง 3.20 กลายเป็นโค้ด 5 บรรทัด

```
const TRANSITIONS = {
  awaiting_dispatch: ['picking', 'cancelled'],
  picking:           ['awaiting_dispatch', 'cancelled'],
  delivered: [],  cancelled: [],
};
```

`delivered` **ไม่อยู่ในรายการปลายทางที่ตั้งได้** — เกิดได้ทางเดียวคือธุรกรรมตัดสต็อก (ประโยคตัวหนาท้ายตาราง 3.20)

ทำไมมี 2 ช่องทาง — 3.5.2: อุปกรณ์เครือข่ายบางรุ่นยกระดับเป็น WebSocket ไม่ได้ จึงมี SSE ที่ `/api/events` สำรอง · โทเคนอยู่ในสตริงคำค้นเพราะเบราว์เซอร์ตั้งหัวข้อคำขอของ WebSocket ไม่ได้ (ข้อสังเกตท้าย 3.6)

ทดสอบ — เปิด server จริง ต่อ WebSocket จริง วัดว่าได้รับใน < 3 วินาที (NFR-02) — ได้ในหลักมิลลิวินาที

4.6 `modules/order` (โมดูล 3) — หัวใจของการขาย

คืออะไร — `quote` (คิดยอด + QR ไม่เขียนฐาน) · `create` (ธุรกรรมเดียว แล้วค่อยพิมพ์ + ผลักเข้าคิว) · `getById`

ใส่มาทำไม — UC-03, UC-04 · FR-03 · เขียนตามรูปที่ 3.10 ทีละขั้น เปิดรูปกับโค้ดคู่กันจะเห็นว่าเรียงตรงกัน

รูป 3.10	**โค้ด**
quote → SELECT ราคา → คำนวณ → QR	`priceItems(pool)` + `buildPayload` — ไม่แตะ `orders`
START TRANSACTION → เลขที่ → orders → items → audit → COMMIT	`withTransaction(...)` บล็อกเดียว
พิมพ์ → push → ตอบ	3 บรรทัดหลังบล็อก **นอกธุรกรรม**

การตัดสินใจสำคัญ - ราคาอ่านจากฐานเสมอ หน้าจอส่งมาแค่ (รหัส, จำนวน) → แก้ราคาใน DevTools ไม่มีผล - คำสั่งซื้อถูกเขียนเมื่อชำระแล้วเท่านั้น ไม่มีสถานะตะกร้า (3.7.2 ข้อ 3) - `unit_price` คัดลอกลง `order_items` ตอนสร้าง → ราคาเปลี่ยนแล้วใบเสร็จเก่าไม่เปลี่ยน (3.7.2 ข้อ 1)

บั๊กที่ทดสอบจับได้ — TC-04 ยิง 20 คำสั่งซื้อพร้อมกัน → deadlock ในฐานข้อมูลตอนคำสั่งซื้อแรกของวัน (`FOR UPDATE` บนแถวที่ยังไม่มีได้แค่ gap lock ซึ่งหลายธุรกรรมถือร่วมกันได้) · ทางแก้: ดัก

deadlock เป็นกรณี “ลองใหม่” + ถอยหลังสุ่ม — ตรงกับ UC-03 ทางเลือกที่ยังสำเร็จในตาราง 3.9 คำต่อคำ

คำถามที่อาจารย์น่าจะถาม — “รู้ได้ไงว่าเลขไม่ซ้ำตอนคนกดพร้อมกัน” → 3 ชั้น: `FOR UPDATE` ให้ต่อคิว · `UNIQUE(order_no)` เป็นตาข่าย · ชนแล้วลองใหม่ตาม UC-03 · และมี test ยิง 20 พร้อมกันเป็นหลักฐาน

4.7 `modules/stock-deduction` (โมดูล 6) — 7 ขั้นของ 3.7.3

คืออะไร — ฟังก์ชันเดียว `dispatch(conn, …)` ตรรกะธุรกรรมล้วน ๆ **ไม่รู้จัก req/res**

ใส่มาทำไมต้องแยกจาก dispatch — 3.6 บอกเอง: “สามารถเขียนชุดทดสอบสำหรับกรณีที่มีคำขอเข้ามาพร้อมกันได้โดยเรียกบริการตัดสต็อกตรง ๆ ไม่ต้องจำลองคำขอผ่านเครือข่าย” · test เรียกตรง ๆ 10 ธุรกรรมพร้อมกัน สำเร็จ 1 ล้ม 9

3 ประโยคที่ควรพูดได้คล่อง 1. กันจ่ายซ้ำด้วย `UPDATE orders … WHERE status IN ('awaiting_dispatch','picking')` — affectedRows = 0 แปลว่ามีคนจ่ายไปก่อน นี่คือ optimistic locking ใน 2.1.3 ไม่ต้องจองล่วงหน้า (E1) 2. กันติดลบด้วย `UPDATE products … WHERE stock_qty >= qty` — คนละปัญหากับข้อ 1 จึงกันแยก และฐานข้อมูลมี `CHECK` เป็นชั้นสุดท้าย (E2) 3. ถ้ารายการที่ 2 ไม่พอ รายการที่ 1 ที่หักไปแล้วต้องคืน — เพราะอยู่ในธุรกรรมเดียว rollback ทั้งก้อน

รายละเอียดเล็กที่สำคัญ — `ORDER BY product_id` ตอนดึงรายการ ให้ทุกธุรกรรมล็อกสินค้าลำดับเดียวกัน ถ้าใบ A ล็อก ปูน→เหล็ก ใบ B ล็อก เหล็ก→ปูน จะ deadlock กัน

4.8 `modules/dispatch` (โมดูล 5) — เปลือกบางของโมดูล 6

คืออะไร — รับ `POST /api/orders/:id/dispatch` → `withTransaction(stockDeduction.dispatch)` → แปลงผลเป็น 200/404/409 → **หลัง commit** broadcast + `alert.checkStock`

ใส่มาทำไม — ตาราง 3.13 โมดูล 5: “รับคำขอยืนยันจ่ายสินค้าและควบคุมลำดับการทำงานของธุรกรรม” · ไม่มีตรรกะสต็อกเลย

3.7.3 ขั้น 7 อยู่ที่นี่ — `alert.checkStock` เรียกนอกธุรกรรมโดยตั้งใจ: ถ้าสร้างแจ้งเตือนล้ม การจ่ายที่เกิดจริงแล้วต้องไม่ถูกยกเลิก (2.1.2 ขอบเขตธุรกรรม)

4.9 `modules/alert` (โมดูล 8) — แจ้งเตือนเชิงรุก

คืออะไร — `checkStock(productIds)` สร้าง `low_stock` · `checkQueueDelay()` สร้าง `queue_delay` ทุก 60 วินาทีจาก timer · `GET/PATCH /api/notifications`

ใส่มาทำไม — FR-08 · BR-05 “ระบบต้องตรวจเงื่อนไขเองหลังทุกครั้งที่สต็อกเปลี่ยน ไม่รอให้ผู้ใช้มาเปิดดู” (2.1.9)

BR-05 กลายเป็น 3 จุดเรียก — dispatch หลัง commit · product.adjustStock หลัง commit · timer ใน server.js

การตัดสินใจสำคัญ - ไม่สร้างซ้ำขณะที่ใบเดิมยังไม่อ่าน — ไม่งั้นขายปูนทุกครั้งได้ "ปูนใกล้หมด" ใบใหม่ - `reorder_point = 0` = ยังไม่ตั้ง ไม่เตือน — ผู้จัดการต้องตั้งเองผ่าน UC-02 (FR-02) - ดัก error เอง ไม่โยน — งานผลพลอยได้ล้ม ต้องไม่ทำให้งานหลักดูเหมือนล้ม (เหมือน print-client)

ทดสอบ AC-10 — จ่ายของจนต่ำกว่าเกณฑ์ → ตรวจฐานทันที**โดยไม่เรียก GET อะไรเลย** → มีแจ้งเตือน

4.10 `modules/dashboard` (โมดูล 7) — คำนวณสด

คืออะไร — `ops` (ยอดวันนี้ · คิวค้าง · สินค้าต่ำกว่าจุดสั่งซื้อ) และ `exec` (แนวโน้มรายวัน · สินค้าขายดี) — เนื้อหาตามตาราง 3.21

ใส่มาทำไม — FR-07 · BR-04 "ตัวชี้วัดต้องคำนวณสดจากฐานข้อมูล ไม่ใช้ค่าสรุปที่เก็บไว้ล่วงหน้า" (2.1.8)

คำถามที่อาจารย์น่าจะถาม — "ไม่แคชแล้วช้าไหม" → ร้านมีคำสั่งซื้อวันละหลักสิบ SUM ทั้งตารางใช้เวลามิลลิวินาที ยอมแลกความเร็วกับความถูกต้อง · test AC-09 เทียบตัวเลขบนแดชบอร์ดกับ SUM ที่ยิง SQL ตรง ๆ ต้องเท่ากัน

---

5. `tests/` — ทำไมทดสอบเป็นส่วนหนึ่งของหลังบ้าน

คืออะไร — `unit/` (ตรรกะล้วน ไม่แตะฐาน: promptpay, print-client, escpos) และ `integration/` (ผ่าน HTTP + ฐาน `soml_new_test` จริง) รวม 93 + 34 ข้อ ทุกข้อชื่อเป็นภาษาไทยและอ้างรหัส AC/TC/หัวข้อในเล่ม

ใส่มาทำไม — ตาราง 3.22 กำหนด 3 ระดับ สองระดับแรกอยู่ที่นี่ · แต่เหตุผลจริงคือ **ทุกการตัดสินใจในเล่มมี test ที่พิสูจน์** ถ้าอาจารย์ถาม "รู้ได้ไงว่า..." คำตอบคือชี้ test

ทำไมใช้ฐานทดสอบแยก — `config/db.js` เลือก `soml_new_test` เมื่อ `NODE_ENV=test` และ `db/reset-db.sh` ตัดคำสั่ง `USE` ออกก่อนโหลด → การรีเซ็ตฐานทดสอบทับฐานจริงไม่ได้

ทดสอบ	พิสูจน์อะไรในเล่ม
`db.test.js`	rollback จริง (2.1.2) · DECIMAL เป็นตัวเลข
`auth.test.js`	TC-01 · AC-01 · ข้อความล็อกอินล้มเหลวเหมือนกันทุกกรณี
`product.test.js`	AC-02 · BR-02 (`stock_qty` = `balance_after` เสมอ)
`order.test.js`	TC-03 ยอดตรง · TC-04 20 คำสั่งซื้อพร้อมกันเลขไม่ซ้ำ · UC-03 ทางที่ล้มเหลว
`dispatch.test.js`	AC-07 (E1 ผ่าน HTTP และเรียกตรง 10 ธุรกรรม) · AC-08 (E2 + หลายรายการ) · ค่าขอบเขต
`queue.test.js`	ตาราง 3.20 ทุกลูกศร · WebSocket/SSE ได้รับใน < 3 วิ (NFR-02)
`alert-dashboard.test.js`	AC-10 ไม่ต้องเปิดดู · AC-09 เทียบกับ SQL ตรง

---

6. คำถามรวมที่ควรซ้อมตอบ

“ทำไมโมดูล 6 กับ 10 ไม่มีผู้ใช้เรียกโดยตรง” — โมดูล 6 (stock-deduction) แยกเพราะเป็นตรรกะที่อ่อนไหวที่สุด รวมไว้ที่เดียวทดสอบกรณีชนกันได้โดยเรียกตรง ๆ · โมดูล 10 (print-client) แยกเพราะเป็นส่วนเดียวที่ผูกกับอุปกรณ์ (3.4.1 พูดไว้ทั้งสองข้อ)

“อะไรอยู่ในธุรกรรม อะไรอยู่นอก” — ใน: สิ่งที่ถ้าล้มแล้วต้องไม่มีอะไรเกิดเลย (orders, order_items, stock, movements, audit) · นอก: งานผลพลอยได้ที่ล้มได้โดยไม่ทำให้งานหลักผิด (พิมพ์, ผลักเข้าคิว, แจ้งเตือน) — 2.1.2

“ถ้าผมแก้ราคาสินค้า ใบเสร็จเก่าเปลี่ยนไหม” — ไม่ เพราะ `order_items.unit_price` เป็น snapshot (3.7.2 ข้อ 1) test “ราคาในใบเสร็จเก่าไม่เปลี่ยน” พิสูจน์ไว้

“สองเครื่องกดจ่ายพร้อมกันเกิดอะไร” — สำเร็จ 1 ปฏิเสธ 1 สต็อกหักครั้งเดียว ด้วย `UPDATE … WHERE status IN (…)` ไม่ใช่ด้วยการล็อกหน้าจอ (AC-07 · TC-06)

“หน้าจอซ่อนเมนูแล้วปลอดภัยหรือ” — ไม่ เมนูหายแค่ให้ใช้ง่าย ความปลอดภัยอยู่ที่ `app.js` + middleware ตอบ 403 (TC-01 ยิงตรงด้วยโทเคนพนักงานขาย)

“ทำไมไม่มีตาราง customers” — ลูกค้าไม่มีบัญชีและไม่กระทำการกับระบบ (3.5.1) แค่สแกนกับรับใบเสร็จ จึงไม่ต้องจำ